

CS112: Friday 11 September, 12:15 to 1:20

DRAFT, RECONSTRUCTED FROM COURSE-MAP.YML AND THE TEACHING LOG

Wednesday we put two addresses on the screen and watched a function change the caller's variable when its parameter was declared with an `&`, and fail to when it was not.

Today: name the three ways, and decide which one you mean. The `&` is not a trick for making functions work. It is a statement about which direction the data is travelling, and C++ puts that statement in every parameter you write.

PART

1. Defining, declaring, and why order matters
2. Prototypes and the `.h` / `.cpp` split
3. The three ways to pass
4. Overloading and default arguments

Part 1. Defining, declaring, and why order matters

The compiler reads top to bottom, once. A function called before it has been seen is an error, which is why the order of functions in a file is not free.

Part 2. Prototypes and the `.h` / `.cpp` split

```
// dice.h      what exists
int roll(int sides);

// dice.cpp    how it works
int roll(int sides) { ... }

// main.cpp    #include "dice.h", then just call it
```

The header is a promise; the `.cpp` is the delivery. Monday's link error was exactly that promise going unkept. It was declared in the header and never written.

Part 3. The three ways to pass

```
void f(int n);           // by value: a copy. Cheap for an
int.                    //
void g(int &n);          // by reference: the variable itself.
void h(const int &n);    // by const reference: no copy, no
changes.
```

Talk to your neighbor. You are passing an object that holds a thousand numbers, and the function only reads it. Which one, and why?

The third, and both halves matter: no copy because copying a thousand numbers to read them is waste, `const` because the function has just been handed the caller's actual data and should promise not to touch it.

Then the vocabulary, which the assignments use: **in** parameters come in, **out** parameters go back, **in/out** do both.

Part 4. Overloading and default arguments

Two functions may share a name if their parameters differ, because the compiler picks by the types at the call. Default arguments collapse several similar calls into one. Both are convenience; neither changes what a call does.